

Java ClassLoader와 Thread Context ClassLoader

PostForge PromptResourceLoader를 이해하기 위한 정리

1. 결론부터

각 스레드는 contextClassLoader 참조를 가질 수 있지만, 스레드마다 반드시 서로 다른 ClassLoader를 사용하는 것은 아니다. 일반적인 단일 Spring Boot 애플리케이션에서는 대부분 같은 ClassLoader를 공유한다.

Thread.currentThread().getContextClassLoader()는 동시성 제어가 아니다. 현재 스레드의 실행 문맥에서 사용할 ClassLoader를 선택하는 API다.

2. 두 종류의 ClassLoader 참조

PromptResourceLoader.class.getClassLoader()

PromptResourceLoader 클래스를 실제로 정의한 ClassLoader다. 애플리케이션 실행 중 사실상 고정되어 있다.

Thread.currentThread().getContextClassLoader()

현재 스레드에 설정된 ClassLoader 참조다. 부모 스레드에서 상속되며 setContextClassLoader()로 변경할 수도 있다.

3. 이 개념은 왜 생겼나

Java ClassLoader는 일반적으로 부모에게 먼저 로딩을 위임한다. 부모 ClassLoader는 자식 ClassLoader에만 존재하는 애플리케이션 클래스나 리소스를 직접 볼 수 없다.

Bootstrap ClassLoader

↓

Platform ClassLoader

↓

Application ClassLoader

↓

WebApp / Plugin ClassLoader

Tomcat 같은 애플리케이션 서버에서는 공통 라이브러리가 상위 ClassLoader에, 각 웹 애플리케이션은 하위 ClassLoader에 놓일 수 있다. 상위 라이브러리가 현재 웹 애플리케이션의 구현체나 리소스를 찾아야 할 때 스레드의 Context ClassLoader를 사용한다.

4. 스레드가 클래스를 직접 로드하는 것은 아니다

스레드는 ClassLoader 자체가 아니라 '현재 실행 문맥에서 우선 사용할 ClassLoader' 참조를 보관한다. 실제 로딩은 그 ClassLoader 객체가 수행한다.

```
thread.setContextClassLoader(webAppClassLoader);
```

이 방식은 애플리케이션 서버, 플러그인, ServiceLoader, JDBC 드라이버 탐색처럼 하나의 JVM 안에 여러 로딩 영역이 존재할 때 유용하다.

5. PostForge 코드에서는

PostForge는 고정된 classpath의 Markdown 프롬프트를 읽는다. 웹앱 또는 플러그인별로 다른 리소스를 선택할 필요가 없으므로 클래스 고정 로더가 의도를 더 정확하게 표현한다.

```
ClassLoader classLoader = PromptResourceLoader.class.getClassLoader();
```

현재처럼 스레드 Context ClassLoader를 사용하면서 캐시는 resourcePath만 키로 삼으면, 서로 다른 ClassLoader가 같은 경로에 다른 리소스를 제공할 때 먼저 읽힌 내용이 전체 캐시에 고정될 수 있다.

6. ClassLoader와 캐시 동시성은 별개다

정적인 MD 파일을 여러 스레드가 동시에 읽는 것 자체는 문제가 아니다. 동시성 고려가 필요한 이유는 여러 요청이 공유하는 변경 가능한 cache Map 때문이다.

```
cache.computeIfAbsent(resourcePath, this::readResource);
```

최초 캐시 미스가 동시에 발생하면 여러 스레드가 같은 키를 계산하고 저장하려고 한다. lazy cache를 유지한다면 ConcurrentHashMap이 필요하다. 캐시를 없애고 매번 독립적으로 파일을 읽는다면 이 공유 상태에 대한 동시성 제어도 필요 없다.

7. 현재 동시성 테스트가 검증하는 것

테스트는 16개 작업 스레드를 start latch 앞에 모은 뒤 동시에 같은 PromptResourceLoader와 같은 resourcePath를 호출한다. 반환 문자열의 내용 비교가 아니라 isSameAs()로 객체 동일성을 확인한다.

```
assertThat(prompts)
    .allSatisfy(prompt -> assertThat(prompt).isSameAs(cachedPrompt));
```

준비 및 완료 순서는 달라도 모든 로그의 cache object ID가 같다면 동시 경합 상황에서 하나의 캐시 값을 공유한다는 뜻이다.

최종 판단

PostForge에서는 PromptResourceLoader.class.getClassLoader()가 적합하다. Thread Context ClassLoader는 필요 이상의 유연성이고, ConcurrentHashMap은 lazy cache라는 공유 상태를 유지하는 동안 필요한 별개의 장치다.